

Hospital Management System: Project Report

Author Name: Shreya Singh

Roll No: 24f3004962@ds.study.iitm.ac.in

Project Overview

In this project, a Hospital Management System (HMS) web application has been developed to address the core challenges faced by hospitals in patient, doctor, appointment, and treatment management. Traditional manual registers and disconnected software solutions create difficulties in record-keeping, scheduling, and tracking patient histories. The HMS streamlines these processes by providing an integrated, role-based solution accessible to Admins, Doctors, and Patients via a modern web interface.

Problem Statement

Many hospitals lack a centralized system to manage records, schedules, and patient histories effectively. This leads to double-booking, loss of patient data, and inefficient administrative workflows. The primary objective of this project was to build a web application to resolve scheduling conflicts, enable secure role-based access, and provide accurate treatment tracking, strictly in accordance with the App Dev I statement requirements.

Frameworks and Technologies Used

- **Backend:** Python Flask

- **Frontend:** Jinja2 templating, HTML, CSS (Custom Glassmorphism Design), Bootstrap 5
- **Database:** SQLite (Programmatically created via SQLAlchemy models)
- **Libraries:** * **Flask-Login** for secure session management and authentication.
 - **Werkzeug** for password hashing and security.
 - **SQLAlchemy** for ORM database interactions.

Note: No external databases (like MySQL/Postgres) or heavy frontend frameworks (like React/Vue) were used, adhering to the project constraints.

Solution Architecture

The application was designed using the **Model-View-Controller (MVC)** paradigm, ensuring separation of concerns and maintainability:

- **Model (`models.py`):** Defines the database schema (User, DoctorProfile, Appointment, Availability) and relationships.
 - **View (`templates/`):** Renders the user interface using HTML and Jinja2, providing dynamic data display.
 - **Controller (`app.py`):** Handles routing, business logic, authentication checks, and database commits.
-

Main Features by Role

Admin (Hospital Staff)

- **Dashboard Analytics:** View real-time statistics of total doctors, patients, and appointments.
- **Doctor Management:** Add new doctors via a popup modal, edit details (fees, department), and delete profiles.
- **Search & Filter:** Advanced filtering of doctors by Name and Department/Specialization.
- **User Control:** Ability to Blacklist/Unblock doctors and patients to prevent unauthorized access.
- **Appointment Oversight:** View a complete history of all hospital appointments with options to cancel or delete records.

Doctor

- **Personalized Dashboard:** View upcoming appointments sorted by date and time.
- **Availability Management:** A visual 7-day calendar allowing doctors to toggle their availability status (Available/Unavailable) for specific dates.
- **Consultation Workflow:** * View patient details.
 - Mark appointments as "Completed" via a modal.
 - Enter Diagnosis, Treatment Plans, and Prescriptions directly into the system.
- **Patient History:** Access the full medical history of assigned patients for informed diagnosis.

Patient

- **Self-Registration:** Secure sign-up and profile management (edit details/password).
 - **Smart Booking System:** * Browse departments with visual banners.
 - View doctor profiles, fees, and 7-day availability status (Green/Red indicators).
 - Book appointments for specific dates and times.
 - **Appointment Management:** * View upcoming schedules.
 - **Reschedule:** Functionality to change appointment date/time if needed.
 - **Cancel:** Option to cancel bookings.
 - **Medical Records:** View past appointment history including Diagnosis and Prescriptions provided by the doctor.
-

Database Design

The database is built on **SQLite** using SQLAlchemy ORM. The schema consists of four primary tables:

1. **User:** Stores login credentials, role (Admin/Doctor/Patient), and contact info. Includes an **is_active_user** flag for blacklisting.
2. **DoctorProfile:** Linked to **User** (One-to-One). Stores specialization, license number, and consultation fees.

3. **Appointment:** Links **Patient** and **Doctor**. Stores date, time, status (Booked/Completed/Cancelled), symptoms, diagnosis, treatment, and prescription.
 4. **Availability:** Tracks specific dates where a doctor has marked themselves as "Unavailable".
-

ER Diagram

The database schema includes the following primary tables:

Note:

Refer to the ER diagram:

[Hospital Management System ER Diagram](#)

APIs and Resource Endpoints

The application follows RESTful conventions using Flask routes:

- **Auth:** `/login`, `/register`, `/logout`
 - **Admin:** * `/admin` (Dashboard & Create Doctor)
 - `/admin/delete/<id>`, `/admin/blacklist/<id>`
 - `/admin/edit/<id>`
 - **Doctor:** * `/doctor` (Dashboard)
 - `/doctor/toggle_availability/<date>`
 - `/doctor/complete_appointment/<id>`
 - **Patient:** * `/patient` (Dashboard & Search)
 - `/book/<doc_id>`, `/patient/reschedule/<id>`
 - `/department/<name>`
-

Application Workflow

1. **Initialization:** The system programmatically creates the database and the default **Admin** account upon the first run.
2. **Setup:** Admin logs in to register Doctors and set their departments.

3. **Patient Journey:** A patient registers, searches for a doctor by specialization, checks availability, and books a slot.
 4. **Consultation:** The Doctor logs in, sees the appointment, treats the patient, fills the diagnosis form, and marks it as complete.
 5. **History:** The appointment moves to the "History" tab for both the Doctor and Patient, containing all medical notes.
-

AI/LLM Declaration

AI and LLM (Language Model) resources were consulted for:

- Code review and error fixing (Python, Flask).
- Guidance on MVC design and database schema relationships.
- Generating sample data descriptions and CSS styling ideas (Glassmorphism).

No direct code generation or copying without modification was utilized; all code was developed, understood, and reviewed before integration.

Demo Video

[Demo Video](#)

Conclusion

The developed Hospital Management System fulfills the requirements laid out in the App Dev I statement. It provides a secure, role-based hospital workflow automation system with robust scheduling, conflict prevention via the availability tracker, and comprehensive record-keeping. The user interface was enhanced with a responsive Glassmorphism design to ensure a high-quality user experience.